

Three weeks, 31 July – 21 August 2026

1. Runtime and language hardening
2. The built-in facet library
3. Comparison with the scientific-pipeline tradition
4. Fleet consolidation, measured
5. Becoming our own Geofabrik

The trap: two sequences

6. From scan to index
7. Making silence audible

Three variants of one root cause

8. What is still open
9. If there is one thing to carry forward

Three weeks, 31 July – 21 August 2026

A retrospective over 100 commits in `facetwork` and 48 across eleven `fwh_*` domain repos. It is written from the git history and the running system rather than from recollection, and it tries to be useful in the way a post-mortem is useful: what changed, what it cost, what was wrong, and what is still open.

One theme runs through almost all of it, and it is not a feature. **The failures that mattered were the ones that did not report themselves.** Every substantial bug found in these three weeks was silent: a workflow that hung with every task complete, an index that answered plausibly with stale data, a map that looked finished while omitting a tenth of its subject, a timer that never fired. Nothing crashed. The recurring lesson is that in a distributed system the dangerous state is not *failed* but *quietly wrong*, and that the only defence is to make each layer say when it has stopped.

1. Runtime and language hardening

The runtime work concentrated on failure semantics — what happens when something goes wrong, and who is allowed to decide.

PermanentError gave handlers a way to say "this will never succeed", which retries had previously been unable to distinguish from "try again". A policy refusal and a network blip had been consuming the same budget.

Cooperative cancellation (`ctx.raise_if_cancelled()`) let a long-running handler stop when its run is terminated, a watchdog has already failed the task, or a reclaim has made it a zombie. A cancelled handler is a clean stop: no retry, no failed step.

Timeout and lease derivation. The lease must outlast the execution timeout, or the owning runner's watchdog resets a wedged task *after* another runner's lease-expiry reclaim has already started running it a second time. Explicit values below that floor are now clamped up with a warning rather than honoured — it is a correctness invariant, not a preference. Paper #5 documents the interaction.

The fan-out stall. Deferred resumes were keyed by workflow when a resume is per *step*, so on a wide fan-out 199 of 200 siblings were dropped. This is the canonical example of the theme: the run did not fail, it simply stopped, with every task complete and nothing to look at. Repair check 7 now detects the resulting stranded state, and the national 3,167-county fan-out was re-run to completion (3,173/3,173 tasks, 0 failed) — surviving a 7.4-hour host hibernation mid-run.

`+=` in a `foreach` `yield`. `yield F(xs = item)` compiles, runs every iteration, and keeps only the last — silently under-reporting a fan-out (3 files became 1 digest). `+=` aggregates. The validator now warns on the bare form for step-derived values.

2. The built-in facet library

`fw.file`, `fw.http`, `fw.archive` and `fw.exec` — 21 facets with no handler to write. The motivation was an audit: 14 of 23 domains had hand-rolled HTTP fetching, one froze on a moving `-latest` target, one recorded a checksum it never compared. Nine domains hand-rolled archive extraction across 17 sites.

The interesting one is `fw.exec`. It takes an argv list and never a shell, and `command` must be a bare name listed in that host's `FW_EXEC_ALLOW` — unset means refuse. The gate exists because the `script {}` sandbox had already decided the FFL author is not fully trusted; without it, `fw.file.WriteText` could plant a binary.

3. Comparison with the scientific-pipeline tradition

A substantial thread compared FFL against Snakemake, Nextflow and Ray, by porting real workflows rather than arguing from features.

- **pypsa-data** — PyPSA-Eur's retrieve layer: **1,631 lines and 73 Snakemake rules collapsed to one `foreach`** over a catalogue, because a rule's outputs are static and the rules are near-copies.
- **pypsa-earth** — the OSM stage: **no collapse at all**, ~1 facet per rule. Three distinct algorithms, 2,168 lines. Exactly what the cost-of-change experiment predicted about `script: -shaped` rules. Porting it also surfaced that upstream's cores are *not actually callable* — two module globals and an attribute access that only Snakemake satisfies.
- **gridbuilder** — the FFL equivalent of PyPSA's grid-builder, calling the same `earth_osm` extractor so the comparison is between orchestrators rather than payloads. Verified same-data: 832 rows, 0 differing.
- **Delegation adapters** for Ray and Snakemake, establishing that an adapter is the polyglot agent protocol pointed at a workflow engine — `claim`, `submit`, `park`, `fw: resume`. Delegation gives reach, **not** portability.

The honest result is mixed, and the paper says so: FFL collapses declarative retrieval dramatically and offers no advantage on algorithmic stages.

4. Fleet consolidation, measured

One runner container per domain is right for a hot domain and poor value for a cold one. Measured before changing anything: **13 of 16 domain runners handled zero tasks in seven days**, each costing ~160 MiB resident and ~8.6 findAndModify/s against MongoDB whether or not it had work.

A generalist runner now fronts the cold domains. A/B, 17 containers vs 4:

	per-domain	consolidated
memory	2,077 MiB	154 MiB (–1.9 GiB)
Mongo ops	175.8/s	41.0/s (–77%)

Per-runner load is $\sim 8.6/s$ in *both* arms, so the saving is proportional to runners removed — predictable rather than a one-off. It needed no new runtime concept: the image already bakes every domain and `–topics` already scopes claiming.

5. Becoming our own Geofabrik

The largest single piece. Phase 1 had split the 87 GB planet into regional extracts and stamped each with a replication URL pointing at us. It never published anything there: every `state.txt` held a bare timestamp with **no sequence number**, and not one `.osc.gz` existed. The indirection was wired to an empty room, so every extract had been frozen for **39 days** — silently.

Phase 2 fills it. Each day's planet diff is cut to every region's polygon and published as that region's own sequenced diff.

Why cutting beats updating the planet, measured: raw disk read is 32 MB/s, so one pass over 87 GB is ~ 45 minutes and a planet-update-plus-re-split cycle runs to hours. Cutting touches only the day's 83 MB diff and never reads the planet. `osmium extract -c` cuts *all* regions in one pass — one region 27.3s, three regions 29.3s — which turned a 39-day catch-up from ~ 2.3 hours into ~ 20 minutes.

The ratio that justifies the whole thing: **europe's 14 days of diffs came to 468 MB against a 37 GB extract**, roughly a thousand to one.

The trap: two sequences

Because diffs are published without being applied, the published *head* runs ahead of the served *extract* by design. Conflating them stamps an extract as newer than it is, so consumers start *after* diffs they still need. This happened: `central-america` was briefly stamped 5053 while its data was at 5051, which would have lost two days of edits for every consumer with no error anywhere. The two are now recorded separately at anchor time, while they are still equal and the fact is still knowable.

6. From scan to index

Overpass remained the primary source for the ALPR map even after the extracts were current, for one reason: it is *indexed*. A selective tag query costs seconds there and a 93 GB scan here.

An incrementally-maintained SQLite index closed that. Built once (76 min), then advanced nightly from the same replication diffs — **23 s/day**. The map now loads **144,635 features in 2.6 s with the network unavailable**.

The rule that makes it honest: an index that only *adds* matches looks perfect and rots. An OSM delete carries no tags, and an untagged node arrives as an ordinary modify that no longer matches. So the rule is per-id and total — `deleted → remove`, `matches → upsert`, `no match → remove`. The last line is easy to omit and impossible to notice.

Building it also corrected a published error: every doc said **~336k** ALPR nodes worldwide. The index produced 144,137; a live Overpass count returned 144,844. The figure was wrong by $2.3\times$ and had apparently never been checked. The two independent measurements agree to 0.14%, the residual being that day's edits. That number appeared in a public map's description.

7. Making silence audible

The last days were spent on the theme itself.

- **fw util ffl-audit** — sweeps every domain repo for FFL and handler-contract drift, and **gates the image bake**. It found a domain whose six handlers had *all* been dead for weeks: they called `params["_step_log"].append(...)` when the runtime passes a callback. Its own test suite stayed green because no test ever injected `_step_log`.
- **fw svc osm-replicate** — nightly publish that verifies the chain afterwards and raises a desktop notification on a stall.
- **fw svc osm-watchdog** — a *separate* agent, because a process cannot observe its own absence. Not hypothetical: a fixed 03:15 nightly entry never fired once, since launchd does not run a missed `StartCalendarInterval` on wake and this machine sleeps.
- **fw maint dead-letters** — the standing question nobody was asking. First sweep: **27 dead letters across 19 facets, oldest 22 days**, none of it known. Including a task that met one transient 503, retried five times, and sat dead for ten days; a single retry completed it in seconds.

Three variants of one root cause

All three scheduled-job failures had the same origin — **launchd hands a job almost no environment**:

1. a bare `python3` resolved to Xcode's sandboxed 3.9, which threw *"Operation not permitted"* on an external volume and lacks every dependency;
2. `Path.glob` returned empty on a denied `readdir`, making an unreadable tree look like an empty one;
3. a bare `osmium` was not on `PATH` at all.

Worth recording: `test -d` and `test -r` both **pass** on a directory you cannot enumerate. They stat, and they lie. `ls` is what tells the truth.

8. What is still open

- **11 dead letters** remain, triaged but not fixed: 4 health maps and h1b blocked on one census lookup (the data is present, so `exists()` likely raises on a 404 instead of returning `False`), a county-atlas osmium failure, an offline `osm.cache` region, 3 Ray delegate jobs, a pypsa-earth array mismatch.
- **The ALPR map is at its rendering cap.** 144,635 features against a 130,000 inline limit, growing ~800/day, and the page is already 58 MB. The sample is now unbiased and declared, but tiled rendering is the real fix.
- **--days catch-up is bounded**, so a long outage needs several runs.
- **Multi-host contention is untested** for wide fan-outs; both acceptance runs were single-host.
- **Cluster hosts are unchanged.** The consolidation and the self-hosted OSM work are per-deployment (`domains.local.json`), so the Intel minis and the infra host are untouched.

9. If there is one thing to carry forward

Four times in three weeks, **a count that did not add up was the only evidence of silently wrong data**: 1325 upserts producing 1256 rows; 144,137 against a claimed 336,000; 3 files yielding 1 digest; 199 of 200 fan-out siblings absent. None surfaced as an error. All were found by noticing arithmetic that did not reconcile.

That is worth more than any individual fix here: when a number is close but not right, it is usually not rounding.